Métodos Internos de Ordenação de Dados

Aula 07 – Bubble Sort, Selection Sort e Insertion Sort



C.E.S.A.R

@CESAR 2021| Todos os Direitos Reservados

Métodos Internos de Ordenação de Dados

- **Bubble Sort;**
- **Selection Sort;**
- **Insertion Sort;**

@CESAR 2021 | Todos os Direitos Reservados



Bubble Sort

Bubble Sort

@CESAR 2021| Todos os Direitos Reservados

- Bubble Sort compara pares adjacentes e troca seus valores se estiverem fora de ordem, repetindo até não precisar mais de trocas.
- **Vantagens**: Simples implementação.
- **Desvantagens**: Ineficiente para grandes conjuntos de dados.
- **Complexidade**: $O(n^2)$

Exemplo:

Array Inicial

Valor	5	1	4	2	8
Índice	0	1	2	3	4

1ª Iteração

Valor	1	4	2	5	8
Índice	0	1	2	3	4

2ª Iteração

Valor	1	2	4	5	8
Índice	0	1	2	3	4

Bubble Sort

@CESAR 2021| Todos os Direitos Reservados

```
#include <iostream>

void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int main() {
    int arr[] = {5, 1, 4, 2, 8}; int tam = 5;
    bubbleSort(arr, tam);
    std::cout << "Array ordenado: ";
    for(int i = 0; i < tam; i++)
        std::cout << arr[i] << " ";
    return 0;
}
```

@CESAR 2021| Todos os Direitos Reservados

Selection Sort

Selection Sort

@CESAR 2021| Todos os Direitos Reservados

- Selection Sort encontra o menor elemento e coloca-o na primeira posição, repetindo este processo até ordenar o array inteiro.
- **Vantagens**: Poucas trocas.
- **Desvantagens**: Complexidade alta para grandes listas.
- **Complexidade**: $O(n^2)$

Exemplo:

Array inicial

Valor	64	25	12	22	11
Índice	0	1	2	3	4

1ª Iteração

Valor	11	25	12	22	64
Índice	0	1	2	3	4

2ª Iteração

Valor	11	12	25	22	64
Índice	0	1	2	3	4

3ª Iteração

Valor	11	12	22	25	64
Índice	0	1	2	3	4

Selection Sort

@CESAR 2021| Todos os Direitos Reservados

```
#include <iostream>

void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int min_idx = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[min_idx])
                min_idx = j;
        }
        int temp = arr[min_idx];
        arr[min_idx] = arr[i];
        arr[i] = temp;
    }
}

int main() {
    int arr[] = {64, 25, 12, 22, 11}; int tam = 5;
    selectionSort(arr, tam);
    std::cout << "Array ordenado: ";
    for(int i = 0; i < tam; i++)
        std::cout << arr[i] << " ";
    return 0;
}
```

@CESAR 2021| Todos os Direitos Reservados

Insertion Sort

Insertion Sort

@CESAR 2021| Todos os Direitos Reservados

- Insertion Sort insere cada elemento em sua posição correta relativa aos elementos já ordenados.
- **Vantagens**: Eficiente para pequenos conjuntos ou quase ordenados.
- **Desvantagens**: Ineficiente para grandes conjuntos aleatórios.
- **Complexidade**: $O(n^2)$

Exemplo:

Array inicial

Valor	12	11	13	5	6
Índice	0	1	2	3	4

1ª Iteração

Valor	11	12	13	5	6
Índice	0	1	2	3	4

2ª Iteração

Valor	11	12	13	5	6
Índice	0	1	2	3	4

3ª Iteração

Valor	5	11	12	13	6
Índice	0	1	2	3	4

4ª Iteração

Valor	5	6	11	12	13
Índice	0	1	2	3	4

Insertion Sort

@CESAR 2021| Todos os Direitos Reservados

```
#include <iostream>

void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

int main() {
    int arr[] = {12, 11, 13, 5, 6}; int n = 5;
    insertionSort(arr, n);
    std::cout << "Array ordenado: ";
    for(int i = 0; i < n; i++)
        std::cout << arr[i] << " ";
    return 0;
}
```

@CESAR 2021| Todos os Direitos Reservados

Exercícios

Bubble Sort

1. Ordene o array [9, 7, 3, 1] usando Bubble Sort.
2. Ordene o array [10, 5, 2, 8, 7] usando Bubble Sort.

Selection Sort

1. Ordene o array [15, 10, 20, 5] usando Selection Sort.
2. Ordene o array [4, 1, 3, 2] usando Selection Sort.

Insertion Sort

1. Ordene o array [6, 4, 3, 7, 1] usando Insertion Sort.
2. Ordene o array [8, 2, 9, 5] usando Insertion Sort.

C . E . S . A . R

Pessoas impulsionando inovação.

Inovação impulsionando negócios.

